Day 42 – HTTP Adapter: CSRF, Certificates, Retry & SFTP Adapter

Class date: 28th August 2026 | Module: Adapters (HTTP & SFTP)

Correction / clarification made to your handwritten notes: You wrote that "CPI don't have a Retry mechanism, but the HTTP Adapter does." This is mostly right but needs one clarification: most CPI adapters genuinely have no automatic retry, and traditionally you had to build retry logic yourself. However, SAP has specifically added a built-in **Retry-on-Exception** feature to the **HTTP Receiver Adapter** (a newer Cloud Integration capability), which is exactly the checkboxes you noted (Retry on Error Response, Retry Interval, Retry Iterations). This only applies to the HTTP receiver adapter, not to CPI as a whole. Explained in full in Section 6 below. Everything else in today's notes – CSRF flow, certificate authentication, SFTP configuration – was accurate.

1. Where we left off – Receiver Adapter settings

Today's focus: the **Receiver Adapter** side of the HTTP Adapter. So far, the following configuration areas of the HTTP Adapter have been covered: **Address**, **Authorization**, **User Role**, and **CSRF Protected**.

2. What is CSRF Protection? (start here first)

CSRF stands for **Cross-Site Request Forgery** (also called a **CSRF attack**). It is a security mechanism, and the checkbox for it is mainly used on the **Receiver** side, though sometimes it is also used on the **Sender** side.

Example used in class – Gmail login: This is mainly about the concept of a **Session ID**. When you log into Gmail, a session ID is created for that login. If someone else somehow gets access to that same session ID, they could hijack your logged-in session and directly use your Gmail account from a different browser, without ever needing your password again. This kind of hijack becomes possible specifically when **CSRF protection is NOT enabled**.

Good news: nowadays CSRF protection is **enabled by default** on most modern web systems and on SAP's own OData sender adapter in CPI. On the HTTP Adapter, however, it is a checkbox you configure explicitly. It is used whenever CPI is connecting to any **third-party secured system** – enable the **CSRF** checkbox under the **HTTPS Connection** tab in that case.

3. Steps to process a CSRF-protected call

Because the target system checks for a valid CSRF token before accepting any data-changing request, the call cannot be completed in a single step. Three steps are followed:

1. **Step 1:** Call the API, passing all required authentication details (Basic/OAuth/certificate, as applicable).
2. **Step 2:** Send the header **X-CSRF-Token: Fetch**. The target system responds with the actual token value inside its **response headers** – remember to look there for it.
3. **Step 3:** Use this fetched token to make the real **POST call** (the actual data-changing call).

Demonstration – iFlow: HttpAdapterFlow

A single call cannot complete the request when CSRF protection is on. Two separate HTTP calls are required:

```
1st call -> GET -> URL, header: X-CSRF-Token: Fetch (Check the response headers after this call - the token value appears there) 2nd call -> POST -> URL, header: X-CSRF-Token: <token from step 1> Result: the real POST call succeeds.
```

Common error if this is done wrong: if the fetched token is missing, expired, or not carried over correctly to the second call, the receiver system returns an **HTTP 403 Forbidden** error.

4. Client Certificate-based Authentication

The client certificate file needs to be **uploaded into the SAP CPI Portal** (the tenant's keystore under Manage Security Material), for it to be used in authentication. This was covered along with **External Certificates** – how and where to add them.

Important rule to remember: when an External Certificate is selected for authentication, the **Authorization** setting in the adapter's Connection tab should be set to **User Role** only (not "Client Certificate" directly maintained at channel level). This is the SAP-recommended approach because certificates mapped via User Role can be renewed/updated without redeploying the iFlow – whereas certificates uploaded directly at iFlow/channel level force a redeploy every time the certificate changes or expires.

Default role: ESBMessaging.send is the SAP-delivered default role used here, same as covered in Day 41 for OAuth-based role restriction.

Best practice noted in class: it is better to store/reference the client certificate at the **Service/keystore level** (centralized, via Security Material and Certificate-to-User mapping) rather than uploading it directly inside the CPI iFlow itself – for the same redeploy-avoidance reason above.

5. Receiver HTTP Adapter – calling a Standard API (demonstration)

Two live demonstrations were done on the Receiver HTTP Adapter:

- The public **Northwind API** (a well-known free OData/REST demo API) was used to demonstrate a generic receiver call.
- SAP's own standard **Message Processing Logs API** was called from the Receiver side, as a second, SAP-specific example.

Under the **Connection** tab of the Receiver HTTP Adapter, the key fields used were:

- **Address** – the target URL.
- **Query** – query parameters appended to the URL, e.g. **$top=1** as a key-value pair (this is an OData system query option that limits the number of records returned – here, to just 1 record).
- **Proxy Type** – whether the call goes via **Internet** (public/cloud endpoint) or **On-Premise** (via SAP Cloud Connector, for backend systems sitting behind a company firewall).
- **Method** – the REST/HTTP verb used for the call, mapped to standard **CRUD** operations (Create = POST, Read = GET, Update = PUT/PATCH, Delete = DELETE).

Other fields also configured: **Send Body** (whether the payload/body is sent with the request), **Authentication** (type used – Basic/OAuth/Certificate), and **Credential Name** (the name of the Security Material entry holding the actual credentials).

6. Retry mechanism on the HTTP Receiver Adapter

Two related but different fields were seen earlier on the receiver adapter: **Private Key Alias**, **Timeout (in ms)** (maximum **60,000 ms / 60 seconds**), and **Streaming**. **Streaming** is used so the adapter sends/receives large payloads in **chunks** instead of loading the entire message into memory at once – this keeps memory consumption low for large files or responses.

Retry, in detail: the default behaviour when a call fails is: **"Throw Exception on Failure"** → this simply throws an error and stops, without retrying the flow. As a general rule, most CPI adapters do not automatically retry a failed call on their own.

However – and this is the correction to your notes – the **HTTP Receiver Adapter specifically** now has a **built-in Retry-on-Exception feature**. The prerequisite is that **"Throw Exception on Failure"** must be enabled first. The related settings are:

- **Attach Error Details on Failure** – when checked, CPI attaches the failed request/response details to the message, useful for debugging in Monitor.
- **HTTP Error Response Code(s)** – the specific response code(s) (e.g. 500, 503) that should trigger a retry.
- **Retry on Exception** – the master checkbox to enable this feature, which then reveals:
- **Retry on Error Response** – enables retrying specifically when an error HTTP status is received.
- **Retry Interval** – how long to wait between attempts, configurable from **5 to 60 seconds**.
- **Retry Iterations** – maximum number of retry attempts, limited to **1 to 3** (to avoid hitting rate limits on the target system).

This feature is specifically valuable for handling **intermittent** failures – e.g. the target server being briefly overloaded – so the iFlow does not need to be manually restarted for a short, temporary glitch.

7. Adapters – Sender / Receiver support (in progress)

A running list of CPI adapters and whether each can be used as Sender and/or Receiver is being built up across classes. So far confirmed:

Adapter	Sender	Receiver
HTTP	Yes	Yes
SFTP	—	—
SOAP	—	—
OData	—	—
Mail	—	—
IDoc	—	—
RFC (XI)	—	—
JDBC	—	—
JMS	—	—
AMQP	—	—

(SFTP, SOAP, OData, Mail, IDoc, RFC(XI), JDBC, JMS, and AMQP entries are still to be filled in as those adapters are covered in upcoming days – they were only listed as adapter names in today's class.)

8. SFTP Adapter – hands-on setup

Practice was done using **files.com**, a real managed SFTP hosting service commonly used for this kind of testing (alongside similar services like ExaVault).

1. **Step 1:** Create a free trial account on files.com (the SFTP test server).
2. **Step 2:** In SAP CPI, go to the **Monitor** tab → open the **Connectivity Test** tile.
3. **Step 3:** Select the **SSH** tab (for testing SFTP/SSH-based connectivity).
4. **Step 4:** Take the **Host Name** from Step 1 (the files.com account) and run the connectivity test.
5. **Step 5:** Copy the **Host Key fingerprint** from the test result and add it to the **known_hosts** file, maintained under **Manage Security Material**.
6. **Step 6:** Go to **Key Store** → **Manage Key Store**. Here you can either create an **SSH key**/key pair, or download the **public key** and open it as an SSH public certificate.

Question raised in class: what is Key-based Authentication? Answer: instead of a username/password, a cryptographic key pair (public + private key) is used to authenticate to the SFTP server. The generated **public key** is shared with the SFTP/receiving team, who register it on their server; CPI then authenticates using the matching **private key** held in its own keystore – nothing secret ever travels over the network.

Manage Security Material – Basic Auth details example given: mail ID and password, just like a Gmail mail ID and password – illustrating that Basic Authentication credentials for SFTP are stored the same simple username/password way as any other Basic Auth entry.

SFTP Adapter – Target tab (Receiver)

Field	Value / Notes
Directory	/output
File Name	File.xml
Append Timestamp	Enabled
Host	SFTP URI – default port 22
Proxy Type	Internet
Authentication	Username or Password
Credential Name	SFTP (the entry created in Manage Security Material)
Timeout	10,000 ms
Maximum Reconnection Attempts	3
Reconnect Delay	1,000 ms
Automatically Disconnect	Enabled – reduces connection failures to the SFTP server
Enable Support for Deprecated Algorithms	Available if the target server requires older SSH algorithms

SFTP Adapter – Processing tab (Receiver)

- **Change Directory Stepwise** – enabled; changes directory levels one at a time rather than jumping straight to the deepest path.

- **Create Directories** – enabled; creates any missing folder levels automatically if they don't already exist on the server.
- **Pattern File Names** – not enabled in this demo; would allow wildcard/pattern-based file naming.
- **Prevent Directory Traversal** – not enabled in this demo; when on, blocks attempts to write outside the configured target directory (a security safeguard).
- **Use Fast Exists Check** – enabled; performs the file-exists check directly on the SFTP server for better performance (falls back to a client-side check if the server doesn't support it).

Handling for Existing Files – defines what happens if a file with the same name already exists at the target. Four options are available:

1. **Override** – replaces the existing file with the new one.
2. **Append** – adds the new content to the end of the existing file.
3. **Fail** – stops processing and raises an error if the file already exists.
4. **Ignore** – skips writing the new file if one already exists at that location.

Quick-recall summary

- **CSRF** = Cross-Site Request Forgery protection; needs a 2-call pattern: GET/Fetch token → POST with token; missing/expired token → **403 Forbidden**.
- Client certificate auth: upload cert to keystore, but set Authorization = **User Role** (not direct cert) to avoid redeploys; default role **ESBMessaging.send**.
- Standard API demo fields: Address, Query ($top=1 etc.), Proxy Type (Internet/On-Premise), Method (CRUD-mapped).
- Most CPI adapters have no auto-retry, **but the HTTP Receiver Adapter does** (Retry Interval 5–60 sec, Retry Iterations 1–3, needs "Throw Exception on Failure" enabled first).
- SFTP setup flow: create test account → Connectivity Test (SSH tab) → host key fingerprint → known_hosts in Security Material → SSH key pair in Key Store.
- SFTP "Handling for Existing Files": **Override, Append, Fail, Ignore.**